

Taller

Josué Fraga

36466

Fundamentos de la programación

Electrónica y Telecomunicaciones

23 de octubre del 2026

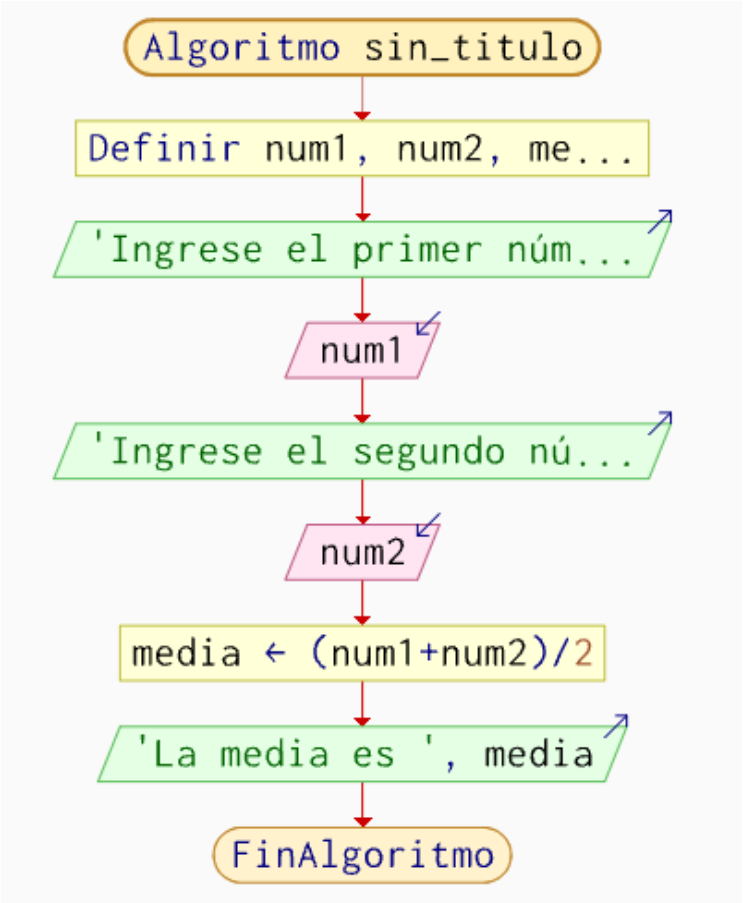
Problema 1.1. Media de dos valores
Análisis del problema
Entrada: 2 variables de tipo real
Proceso: suma de variables y división entre 2
Salida: Imprimir en pantalla la media de los dos números
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Num1	Num1	Var	Real
Num2	Num2	Var	Real
media	media	Constante	Entero

Seudocódigo

```
1  Algoritmo Media
2      definir num1, num2, media Como Real
3      escribir "ingrese el primer numero"
4      leer num1
5      escribir "Ingrese el segundo numero"
6      leer num2
7      media ← (num1 + num2) / 2
8  FinAlgoritmo
9
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>

int main() {
    float num1, num2, media;

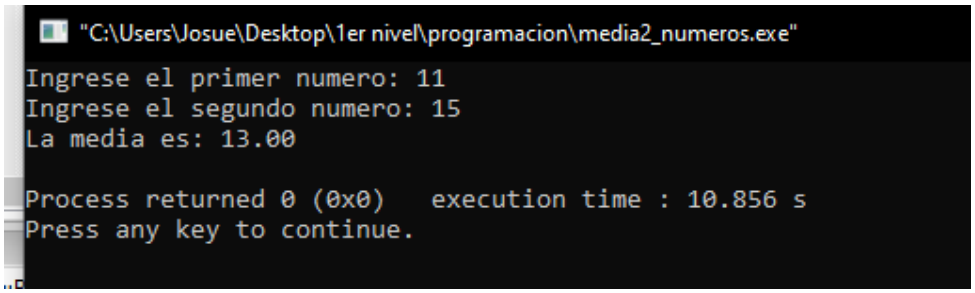
    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

    media = (num1 + num2) / 2;

    printf("La media es: %.2f\n", media);
}
```

Ejecución



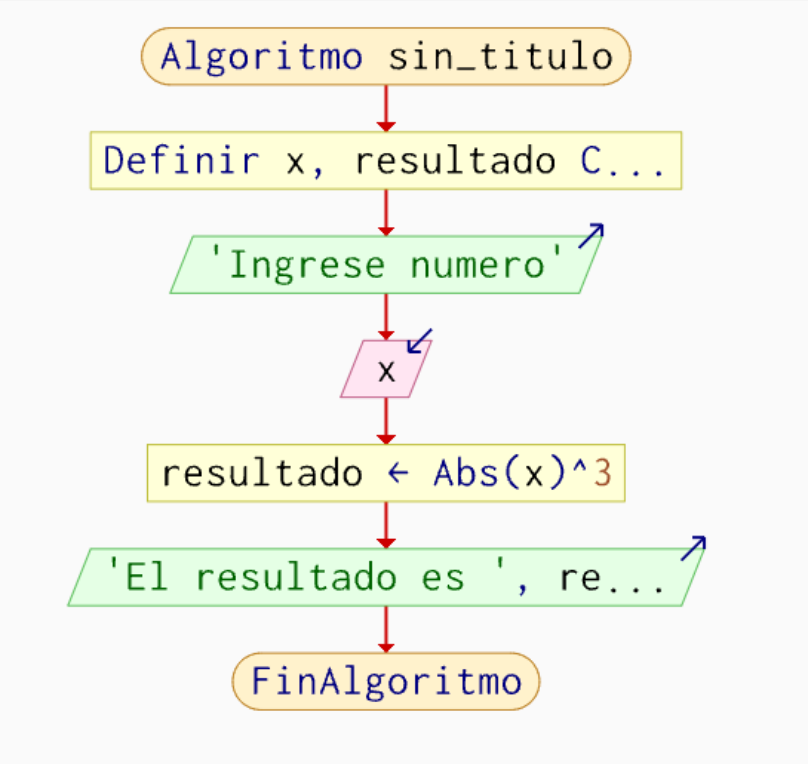
Problema 1.2. Valor absoluto de x al cubo
Análisis del problema
Entrada: X como numero real
Proceso: multiplicar |x| tres veces
Salida: Imprimir en pantalla el numero siempre positivo
Tabla de objetos

Objeto	Nombre	Valor	Tipo
X	X	Var	Real
Resultado	Resultado	Var	Real
Absx^3	Operador	Constante	Real

Seudocódigo

```
1  Algoritmo sin_titulo
2      Definir x, resultado Como Real
3      Escribir "Ingrese numero"
4      Leer x
5      resultado ← Abs(x)↑3
6      Escribir "El resultado es ", resultado
7  FinAlgoritmo
8
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
#include <math.h>

int main() {
    float x, resultado;

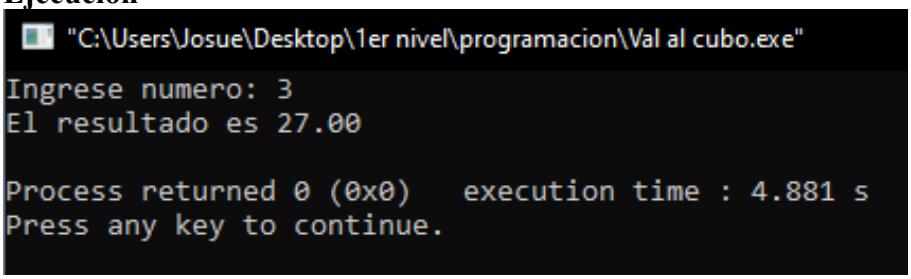
    printf("Ingrese numero: ");
    scanf("%f", &x);

    resultado = pow(fabs(x), 3);

    printf("El resultado es %.2f\n", resultado);

    return 0;
}
```

Ejecución



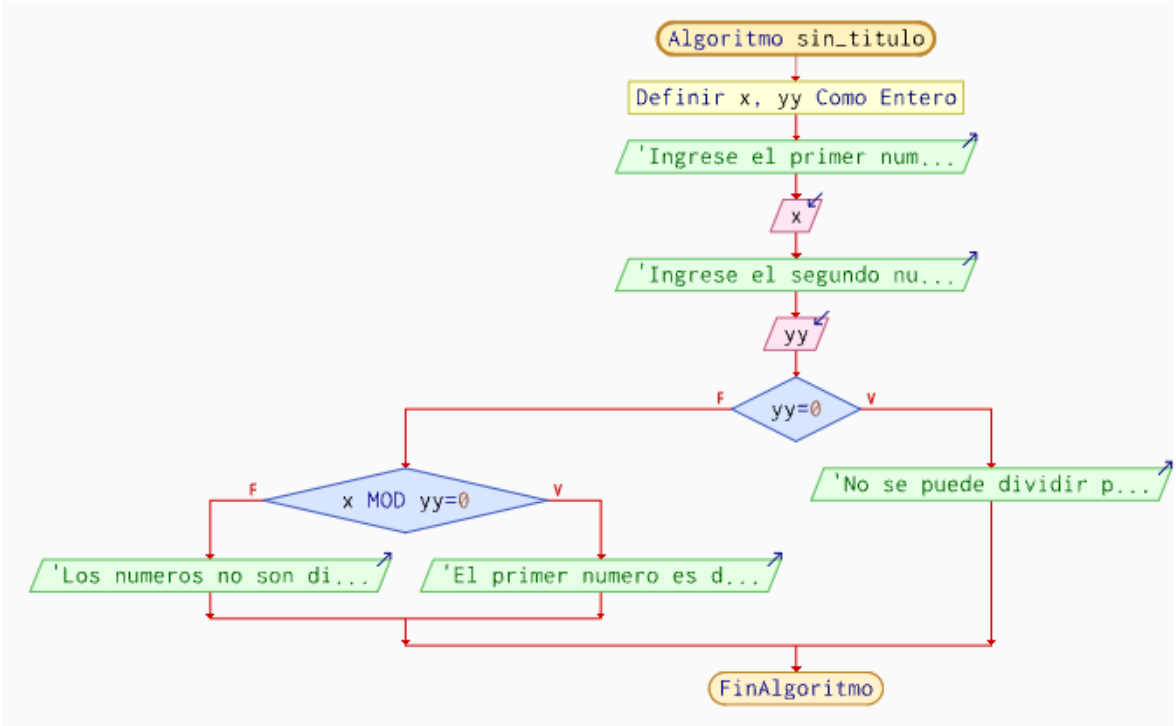
Problema 1.3. Divisible
Análisis del problema
Entrada: 2 números enteros
Proceso: analizar si el residuo de la división es igual a 0
Salida: Imprimir en pantalla el numero es divisible si no, no es divisible
Restricciones: El segundo numero no puede ser 0
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero1	X	Var	Real
Numero divisor	Yy	Var	Real
mod	Operador	Constante	Real

Seudocódigo

```
Algoritmo sin_titulo
    Definir x, yy Como Entero
    Escribir "Ingrese el primer numero"
    Leer x
    Escribir "Ingrese el segundo numero"
    Leer yy
    Si yy = 0 Entonces
        Escribir "No se puede dividir para cero"
    Sino
        Si x % yy = 0 Entonces
            Escribir "El primer numero es divisible por el segundo"
        Sino
            Escribir "Los numeros no son divisibles"
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>

int main() {
    int x, yy;

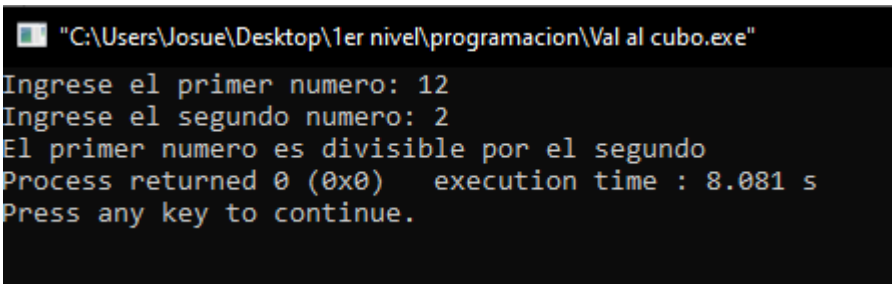
    printf("Ingrese el primer numero: ");
    scanf("%d", &x);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &yy);

    if (yy == 0) {
        printf("No se puede dividir para cero");
    } else {
        if (x % yy == 0) {
            printf("El primer numero es divisible por el segundo");
        } else {
            printf("Los numeros no son divisibles");
        }
    }

    return 0;
}
```

Ejecución



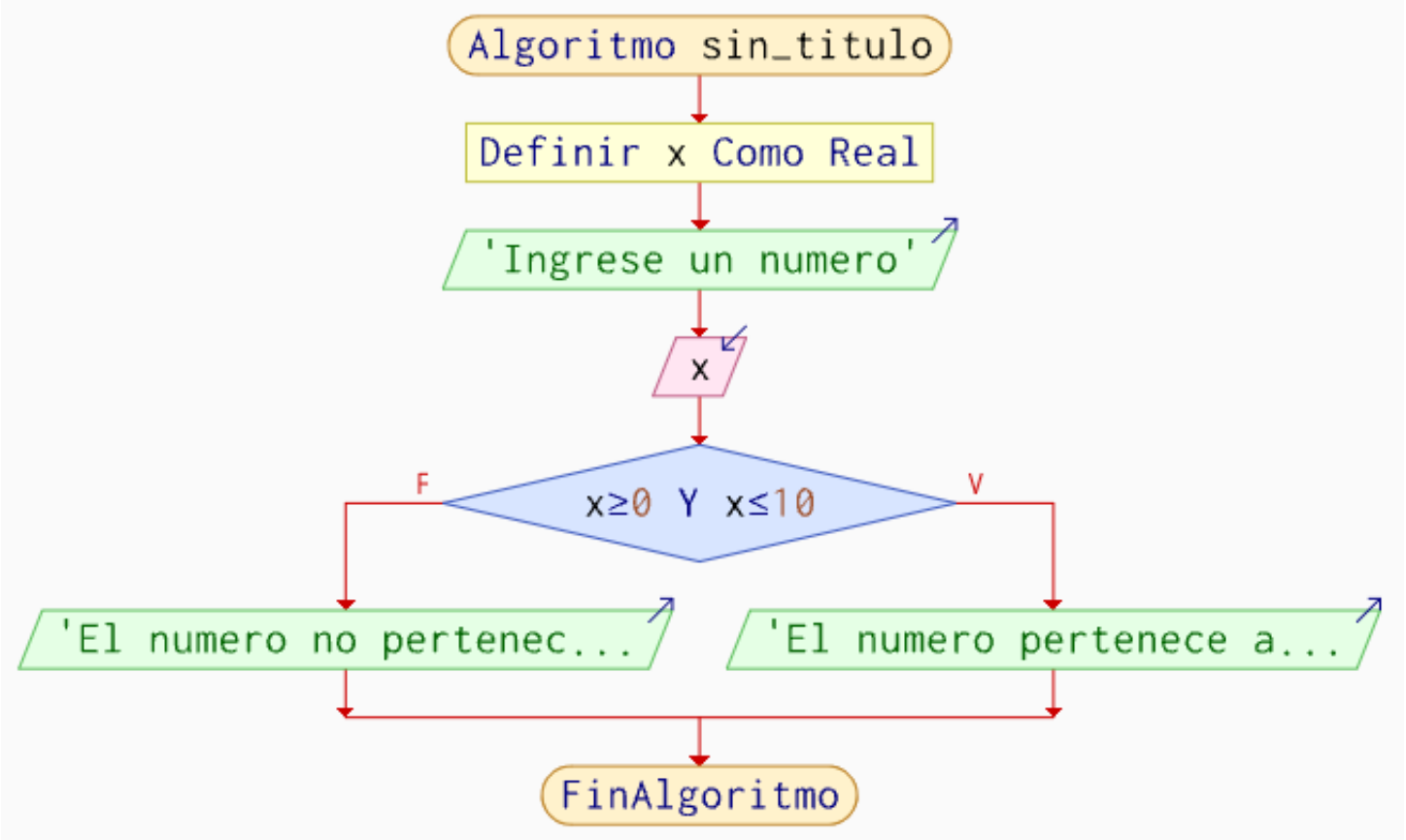
Problema 1.4. Intervalo Análisis del problema
Entrada: Un numero X tipo real
Proceso: Evaluar mediante if si esta en el intervalo de 0 a 10
Salida: Imprimir en pantalla si el numero esta en dicho intervalo
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero inresado	X	Var	Real

Seudocódigo

```
Algoritmo sin_titulo
    Definir x Como Real
    Escribir "Ingrese un numero"
    Leer x
    Si x ≥ 0 Y x ≤ 10 Entonces
        Escribir "El numero pertenece al intervalo [0, 10]"
    Sino
        Escribir "El numero no pertenece al intervalo [0, 10]"
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>

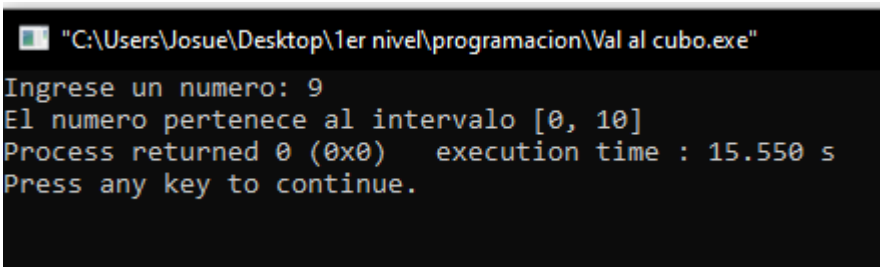
int main() {
    float x;

    printf("Ingrese un numero: ");
    scanf("%f", &x);

    if (x >= 0 && x <= 10) {
        printf("El numero pertenece al intervalo [0, 10]");
    } else {
        printf("El numero no pertenece al intervalo [0, 10]");
    }

    return 0;
}
```

Ejecución



Problema 1.5. Conversión de unidades de tiempo
Entrada: c, días, horas, minutos, segundos, como enteros
Proceso: Modulo de c en horas minutos y segundos
Salida: Imprimir en pantalla días horas minutos y segundos
Tabla de objetos

Objeto	Nombre	Valor	Tipo
C	c	Var	Entero
Cont seg	Segundos	Var	Entero
Cont horas	Horas	Var	Entero
Cont min	Minutos	var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir c, dias, horas, minutos, segundos Como Entero
    Escribir "Ingrese la cant de segundos"
    Leer c
    dias ← trunc(c / 86400)
    c ← c MOD 86400
    horas ← trunc(c / 3600)
    c ← c MOD 3600
    minutos ← trunc(c / 60)
    segundos ← c MOD 60
    Escribir "Dias ", dias
    Escribir "Horas ", horas
    Escribir "Minutos ", minutos
    Escribir "Segundos ", segundos
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>

int main() {
    int c, dias, horas, minutos, segundos;

    printf("Ingrese la cantidad de segundos: ");
    scanf("%d", &c);

    dias = c / 86400;
    c = c % 86400;

    horas = c / 3600;
    c = c % 3600;

    minutos = c / 60;
    segundos = c % 60;

    printf("Dias: %d\n", dias);
    printf("Horas: %d\n", horas);
    printf("Minutos: %d\n", minutos);
    printf("Segundos: %d\n", segundos);

    return 0;
}
```

Ejecución

```
Ingrese la cantidad de segundos: 1000000
Dias: 11
Horas: 13
Minutos: 46
Segundos: 40

Process returned 0 (0x0)   execution time : 3.028 s
Press any key to continue.
```

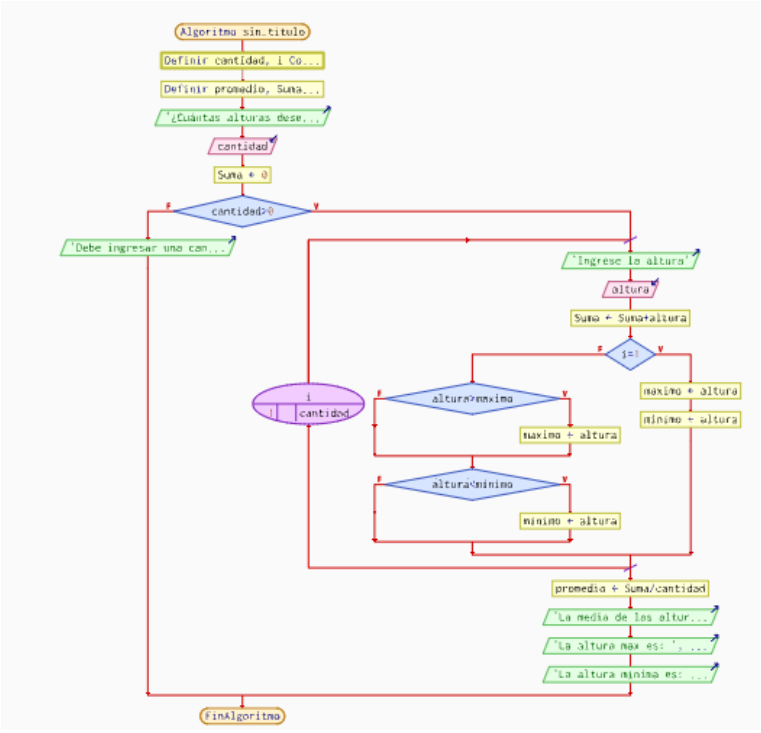
Problema 1.6. Máximo, mínimo y media de alturas
Entrada: secuencia de alturas en números reales no se permiten negativos
Proceso: para cada altura se valida y se actualiza el contador n
Salida: media, altura mas y min
Restricciones: las alturas no pueden ser negativas
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Alturas	cant	Var	real
Cant alturas	i	Var	Entero
Prom alturas	Promedio	Var	real
Alt max	Max	Var	Real
Alt min	Min	Var	Real

Seudocódigo

```
Algoritmo sin_titulo
    Definir cantidad, i Como Entero
    Definir promedio, suma, altura, maximo, minimo Como Real
    Escribir "¿Cuántas alturas desea ingresar?"
    Leer cantidad
    Suma ← 0
    Si cantidad > 0 Entonces
        Para i ← 1 Hasta cantidad Hacer
            Escribir "Ingrese la altura"
            Leer altura
            Suma ← Suma + altura
            Si i = 1 Entonces
                maximo ← altura
                minimo ← altura
            Sino
                Si altura > maximo Entonces
                    maximo ← altura
                FinSi
                Si altura < minimo Entonces
                    minimo ← altura
                FinSi
            FinSi
        FinPara
        promedio ← Suma / cantidad
        Escribir "La media de las alturas es: ", promedio
        Escribir "La altura max es: ", maximo
        Escribir "La altura mínima es: ", minimo
    Sino
        Escribir "Debe ingresar una cantidad mayor que 0"
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int cantidad = 0;
    float promedio, suma = 0, altura, maximo, minimo;

    printf("Ingrese una altura (negativa para terminar): ");
    scanf("%f", &altura);

    while (altura >= 0) {
        suma = suma + altura;
        if (cantidad == 0) {
            maximo = altura;
            minimo = altura;
        } else {
            if (altura > maximo) {
                maximo = altura;
            }
            if (altura < minimo) {
                minimo = altura;
            }
        }
        cantidad = cantidad + 1;
        printf("Ingrese una altura (negativa para terminar): ");
        scanf("%f", &altura);
    }
    if (cantidad > 0) {
        promedio = suma / cantidad;
        printf("La media de las alturas es: %.2f\n", promedio);
        printf("La altura maxima es: %.2f\n", maximo);
        printf("La altura minima es: %.2f\n", minimo);
    } else {
         printf("No se ingresaron alturas validas.\n");
     }

     return 0;
}
```

Ejecución

```
Ingrese una altura: 45
Ingrese una altura (negativa para terminar): 78
Ingrese una altura (negativa para terminar): 10
Ingrese una altura (negativa para terminar): 0.5
Ingrese una altura (negativa para terminar): -5
La media de las alturas es: 33.38
La altura maxima es: 78.00
La altura minima es: 0.50

Process returned 0 (0x0)   execution time : 11.900 s
Press any key to continue.
```

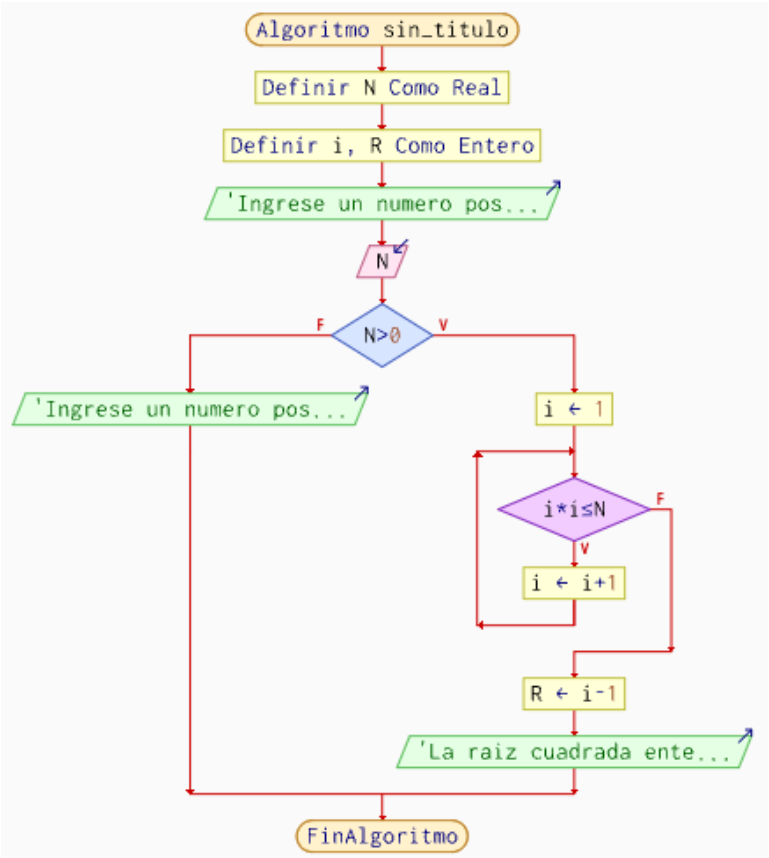
Problema 1.7. Raíz cuadrada entera
Entrada: N, i, R como enteros
Proceso: Ingresar un numero positivo e iniciar el contador i en uno, multiplicar hasta que este sea menor igual que N, luego almacenar este resultado en R restándole uno para que sea inmediatamente la raíz del numero que ingresamos
Salida: Raiz del numero postivo N
Restricciones: No numero negativos
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero positivo	N	Var	Entero
Real	R	Var	Entero
Cont	i	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir N Como Real
    Definir i, R Como Entero
    Escribir "Ingrese un numero positivo:"
    Leer N
    Si N > 0 Entonces
        i ← 1
        Mientras i * i ≤ N Hacer
            i ← i + 1
        FinMientras
        R ← i - 1
        Escribir "La raiz cuadrada entera es: ", R
    SiNo
        Escribir "Ingrese un numero positivo"
    FinSi
FinAlgoritmo
```

Diagrama de flujo

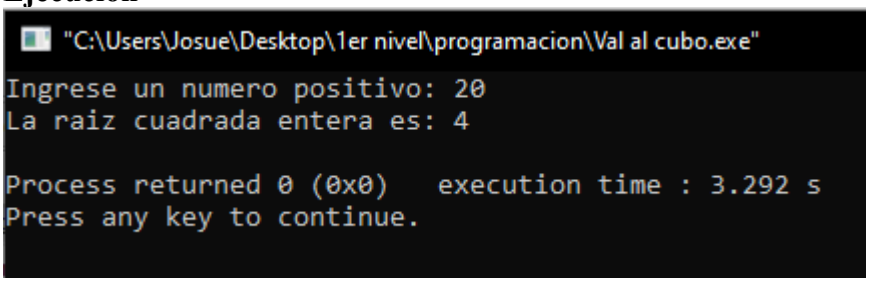


Código fuente

```c
#include <stdio.h>
int main() {
    float N;
    int i, R;
    printf("Ingrese un numero positivo: ");
    scanf("%f", &N);
    if (N > 0) {
        i = 1;
        while (i * i <= N) {
            i = i + 1;
        }
        R = i - 1;
        printf("La raiz cuadrada entera es: %d\n", R);
    } else {
        printf("Ingrese un numero positivo\n");
    }
    return 0;
}

```

Ejecución



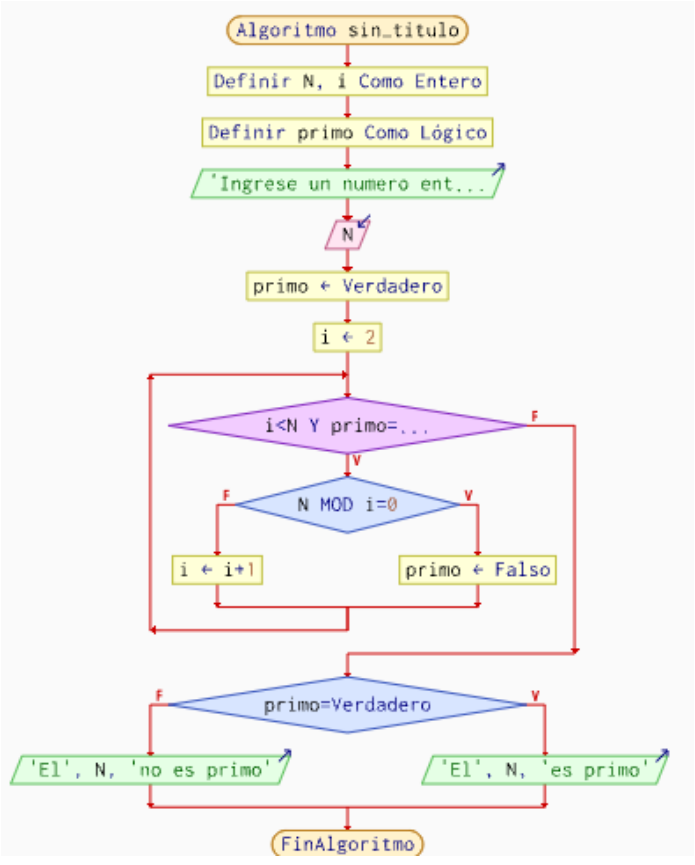
Problema 1.8. Número primo
Entrada: Numero entero n
Proceso: saber si este numero n es divisible solo para dos números
Salida: Imprimir en pantalla el numero es primo
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero positivo	N	Var	Entero
Contador	i	var	Entero
Primo	primo		Lógico

Seudocódigo

```
Algoritmo sin_titulo
    Definir N, i Como Entero
    Definir primo Como Logico
    Escribir "Ingrese un numero entero mayor que 1"
    Leer N
    primo ← Verdadero
    i ← 2
    Mientras i < N Y primo = Verdadero Hacer
        Si N Mod i = 0 Entonces
            primo ← Falso
        Sino
            i ← i + 1
        FinSi
    FinMientras
    Si primo = Verdadero Entonces
        Escribir "El", N ,"es primo"
    Sino
        Escribir "El", N ,"no es primo"
    FinSi
FinAlgoritmo
```

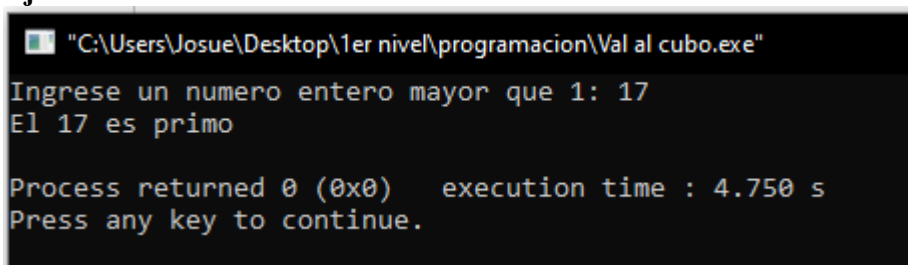
Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int N, i;
    int primo;
    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &N);
    primo = 1;
    i = 2;
    while (i < N && primo == 1) {
        if (N % i == 0) {
            primo = 0;
        } else {
            i = i + 1;
        }
    }
    if (primo == 1) {
        printf("El %d es primo\n", N);
    } else {
        printf("El %d no es primo\n", N);
    }
    return 0;
}
```

Ejecución



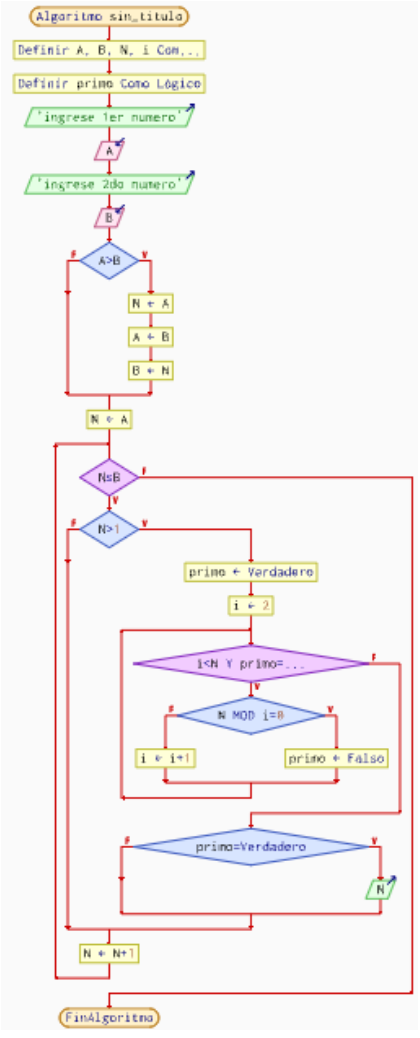
Problema 1.9. Números primos comprendidos entre dos números dados
Entrada: los números que delimitaran el intervalo A, B
Proceso: definimos el intervalo y seleccionamos los numero primos con el operador mod
Salida: Imprimir en pantalla los números primos

Objeto	Nombre	Valor	Tipo
Intervalo	A, B	Var	Entero
Divisor comprobador	i	var	Entero
Numero primo	N	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir A, B, N, i Como Entero
    Definir primo Como Logico
    Escribir "ingrese 1er numero"
    Leer A
    Escribir "ingrese 2do numero"
    Leer B
    Si A > B Entonces
        N ← A
        A ← B
        B ← N
    FinSi
    N ← A
    Mientras N ≤ B Hacer
        Si N > 1 Entonces
            primo ← Verdadero
            i ← 2
            Mientras i < N Y primo = Verdadero Hacer
                Si N MOD i = 0 Entonces
                    primo ← Falso
                SiNo
                    i ← i + 1
                FinSi
            FinMientras
            Si primo = Verdadero Entonces
                Escribir N
            FinSi
        FinSi
        N ← N + 1
    FinMientras
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int A, B, N, i;
    int primo;
    printf("Ingrese 1er numero: ");
    scanf("%d", &A);
    printf("Ingrese 2do numero: ");
    scanf("%d", &B);
    if (A > B) {
        N = A;
        A = B;
        B = N;
    }
    N = A;
    while (N <= B) {
        if (N > 1) {
            primo = 1;
            i = 2;
            while (i < N && primo == 1) {
                if (N % i == 0) {
                    primo = 0;
                } else {
                    i = i + 1;
                }
            }
            if (primo == 1) {
                printf("%d\n", N);
            }
        }
        N = N + 1;
    }
    return 0;
```

Ejecución

```
Ingrese 1er numero: 2
Ingrese 2do numero: 25
2
3
5
7
11
13
17
19
23

Process returned 0 (0x0)   execution time : 4.571 s
Press any key to continue.
```

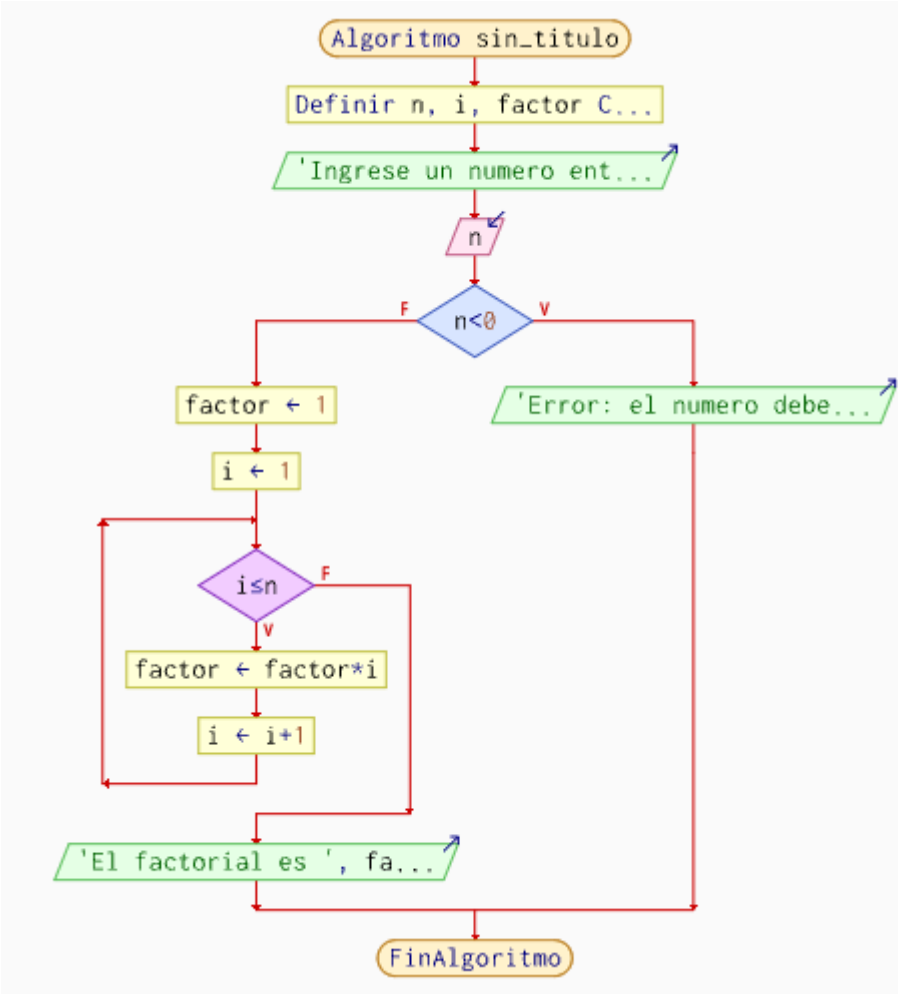
Problema 1.10. Factorial
Entrada: numero entero n mayor o igual a 0
Proceso: iniciar el contador i en 1 y multiplicar el contador i hasta que i sea menor igual que n
Salida: Factorial del numero ingresado

Objeto	Nombre	Valor	Tipo
n	N	Var	Entero
i	I	var	Entero
factorial	factor	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir n, i, factor Como Entero
    Escribir "Ingrese un numero entero mayor igual a 0:"
    Leer n
    Si n < 0 Entonces
        Escribir "Error: el numero debe ser mayor o igual a 0"
    Sino
        factor ← 1
        i ← 1
        Mientras i ≤ n Hacer
            factor ← factor * i
            i ← i + 1
        FinMientras
        Escribir "El factorial es ", factor
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int n, i, factor;

    printf("Ingrese un numero entero mayor o igual a 0: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Error: el numero debe ser mayor o igual a 0");
    } else {
        factor = 1;
        i = 1;

        while (i <= n) {
            factor = factor * i;
            i = i + 1;
        }

        printf("El factorial es %d\n", factor);
    }

    return 0;
}
```

Ejecución

```
Ingrese un numero entero mayor o igual a 0: 5
El factorial es 120

Process returned 0 (0x0)   execution time : 2.901 s
Press any key to continue.
```

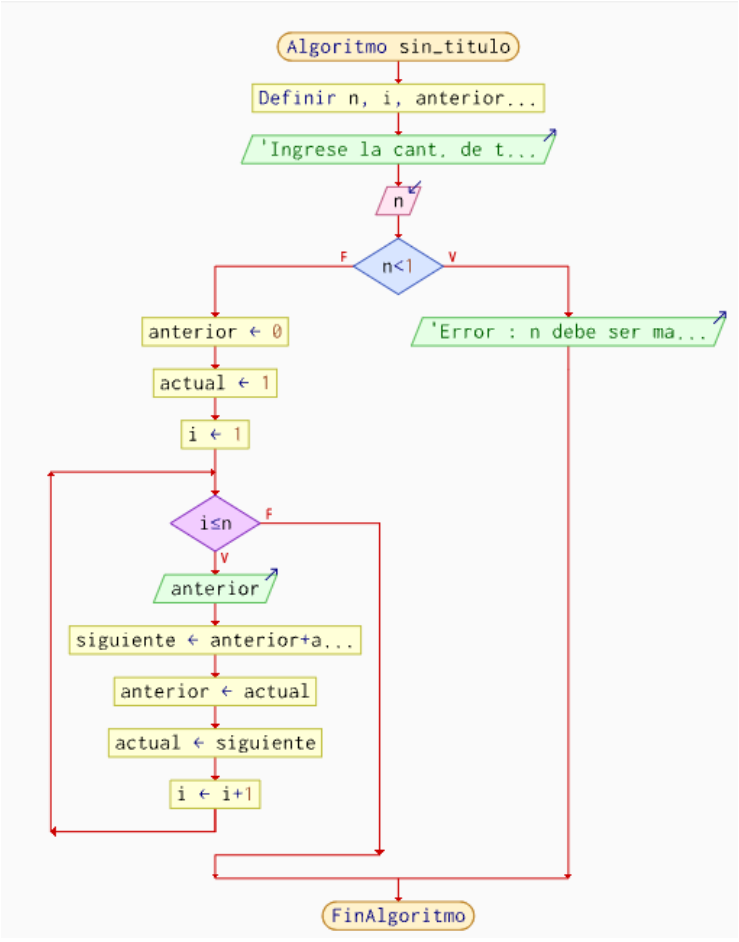
Problema 1.11. Serie de Fibonacci
Entrada: Numero entero mayor q uno que representa la cantidad de números que se mostraran
Proceso: inicializar anterior en 0 y actual en, i=1 mientras i==n
Salida: los n términos de Fibonacci

Objeto	Nombre	Valor	Tipo
n	n	Var	Entero
i	i	var	Entero
Anterior	Ant	Var	Entero
Actual	Act	Var	Entero
Siguiente	sig	var	Entero

Seudocódigo

```
1  Algoritmo sin_titulo
2      Definir n, i, anterior, actual, siguiente Como Entero
3      Escribir "Ingrese la cant. de terminos"
4      Leer n
5      Si n < 1 Entonces
6          Escribir "Error : n debe ser mayor o igual a 1"
7      SiNo
8          anterior ← 0
9          actual ← 1
10         i ← 1
11         Mientras i ≤ n Hacer
12             Escribir anterior
13             siguiente ← anterior + actual
14             anterior ← actual
15             actual ← siguiente
16             i ← i + 1
17         FinMientras
18     FinSi
19 FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int n, i, anterior, actual, siguiente;

    printf("Ingrese la cantidad de terminos: ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Error: n debe ser mayor o igual a 1\n");
    } else {
        anterior = 0;
        actual = 1;
        i = 1;

        while (i <= n) {
            printf("%d\n", anterior);

            siguiente = anterior + actual;
            anterior = actual;
            actual = siguiente;

            i = i + 1;
        }
    }
    return 0;
}
```

Ejecución

```
Ingrese la cantidad de terminos: 7
0
1
1
2
3
5
8

Process returned 0 (0x0)   execution time : 3.035 s
Press any key to continue.
```

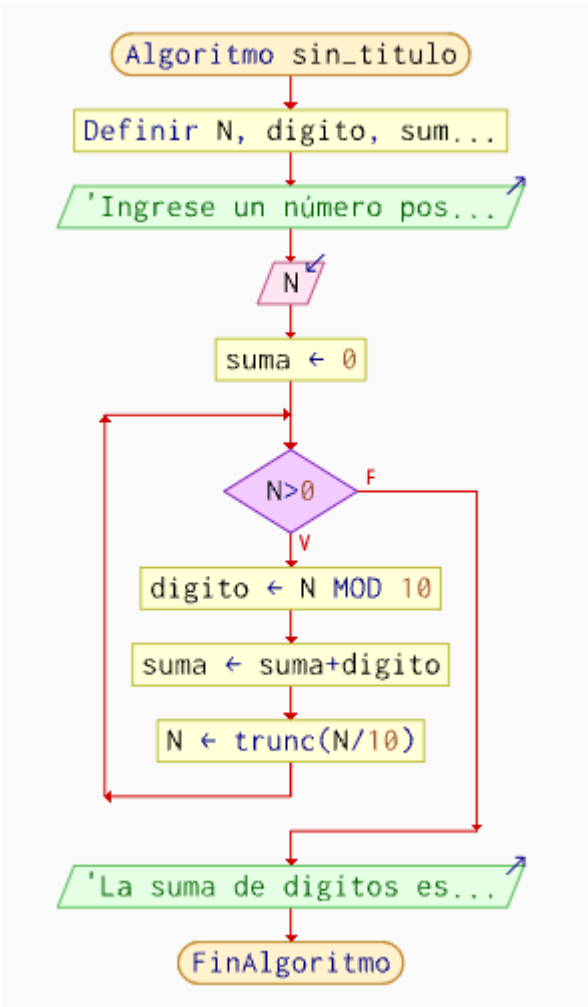
Problema 1.12. Suma de dígitos
Entrada: n, digito, suma, como enteros
Proceso: iniciar la suma en 0 obtener el ultimo digito mediante Nmod10 sumarlo al acumulados y reoetir mientras n mayor que 0
Salida: los n términos de Fibonacci

Objeto	Nombre	Valor	Tipo
n	n	Var	Entero
digito	Digito	var	Entero
Suma	suma	Var	Entero

Seudocódigo

```
1  Algoritmo sin_titulo
2      Definir N, digito, suma Como Entero
3      Escribir "Ingrese un número positivo"
4      Leer N
5      suma ← 0
6      Mientras N > 0 Hacer
7          digito ← N MOD 10
8          suma ← suma + digito
9          N ← trunc(N / 10)
10     FinMientras
11     Escribir "La suma de digitos es ", suma
12 FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c

#include <stdio.h>

int main() {
    int N, digito, suma;

    printf("Ingrese un numero positivo: ");
    scanf("%d", &N);

    suma = 0;

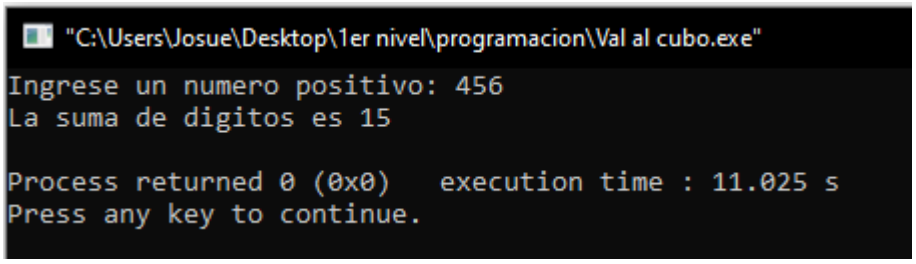
    while (N > 0) {
        digito = N % 10;
        suma = suma + digito;
        N = N / 10;
    }

    printf("La suma de digitos es %d\n", suma);

    return 0;
}

```

Ejecución



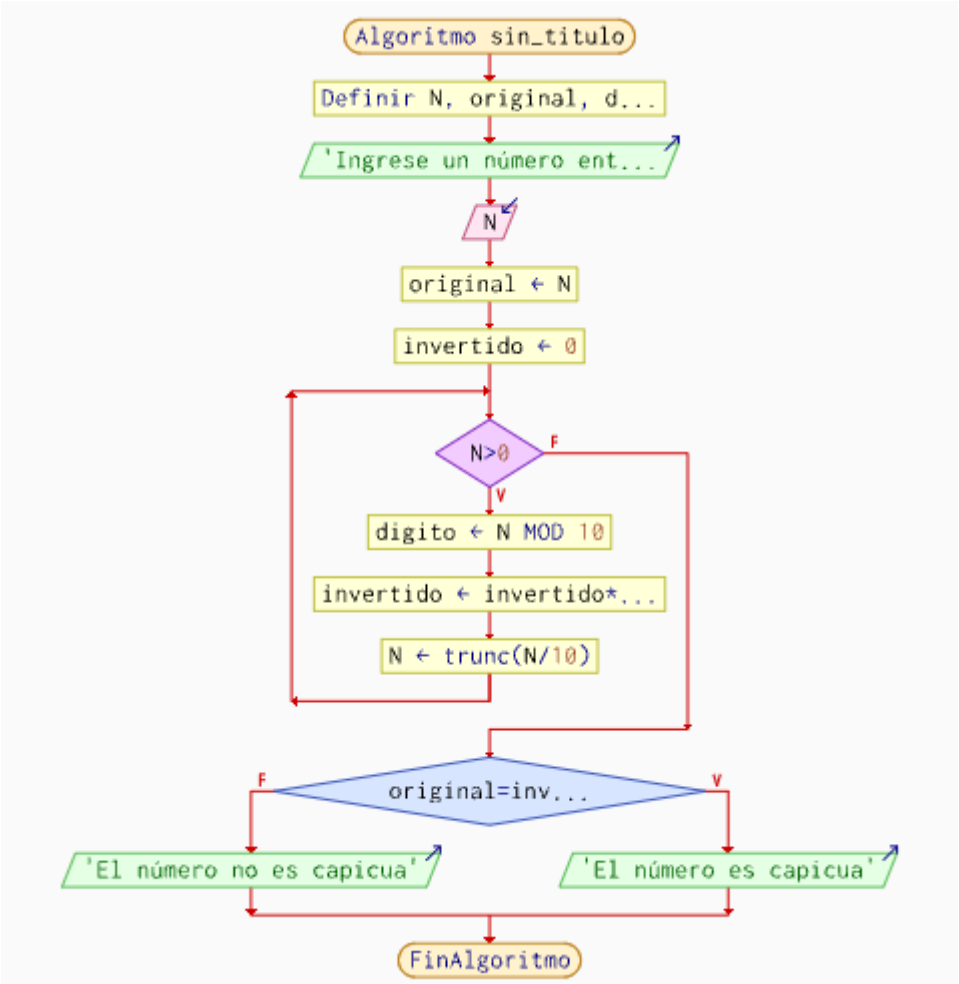
Problema 1.13. Número capicúa
Entrada: un numero positivo ingresado
Proceso: gurda el numero original e inicia el numero invertido en 0 extrae los dígitos con mod10
Salida: mostrar si el numero es capicúa o no

Objeto	Nombre	Valor	Tipo
Original	Original	Var	Entero
digito	Digito	var	Entero
invertido	invertido	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir N, original, digito, invertido Como Entero
    Escribir "Ingrese un número entero positivo"
    Leer N
    original ← N
    invertido ← 0
    Mientras N > 0 Hacer
        digito ← N MOD 10
        invertido ← invertido * 10 + digito
        N ← trunc(N / 10)
    FinMientras
    Si original = invertido Entonces
        Escribir "El número es capicua"
    Sino
        Escribir "El número no es capicua"
    FinSi
FinAlgoritmo
```

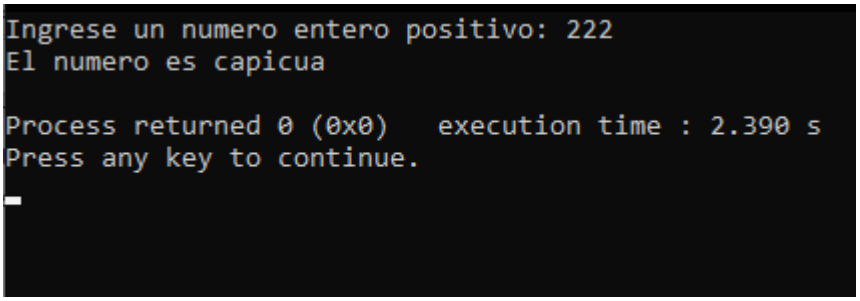
Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int N, original, digito, invertido;
    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &N);
    original = N;
    invertido = 0;
    while (N > 0) {
        digito = N % 10;
        invertido = invertido * 10 + digito;
        N = N / 10;
    }
    if (original == invertido) {
        printf("El numero es capicua\n");
    } else {
        printf("El numero no es capicua\n");
    }
    return 0;
}
```

Ejecución



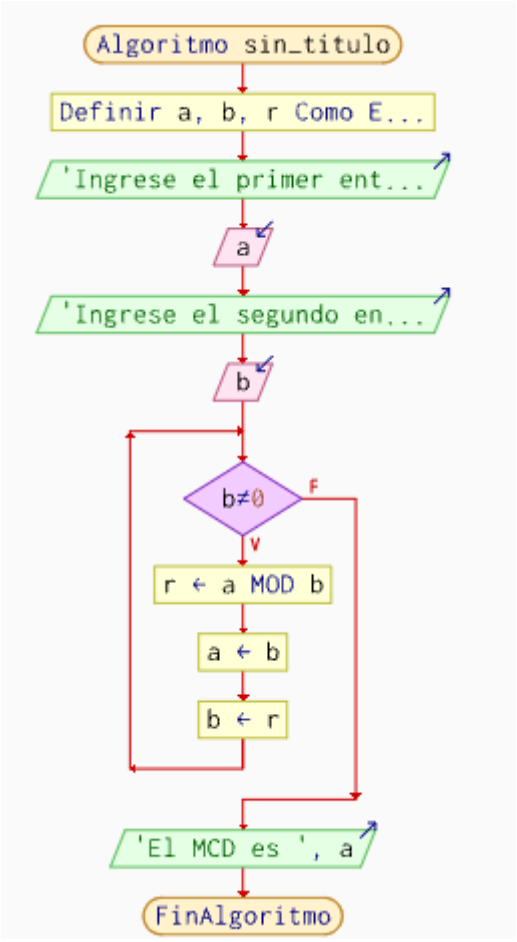
Problema 1.14. Máximo común divisor (algoritmo de Euclides)
Entrada: un numero entero positivo a y un numero positivo b
Proceso: leer los valores a y b mientras b sea diferente de cero calcular el residuo de dividir a para b
Salida: Mostrar en pantalla en MCD de los números

Objeto	Nombre	Valor	Tipo
Primer numero	A	Var	Entero
Segundo numero	B	var	Entero
Residuo	r	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir a, b, r Como Entero
    Escribir "Ingrese el primer entero positivo"
    Leer a
    Escribir "Ingrese el segundo entero positivo"
    Leer b
    Mientras b ≠ 0 Hacer
        r ← a MOD b
        a ← b
        b ← r
    FinMientras
    Escribir "El MCD es ", a
FinAlgoritmo
```

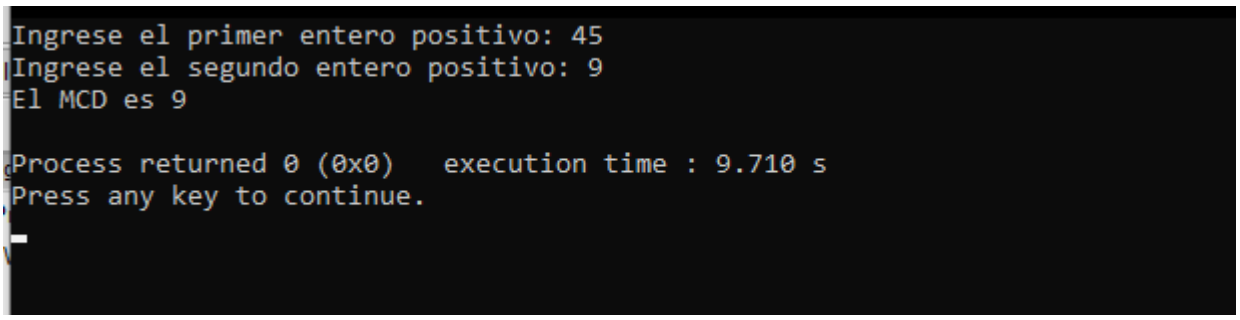
Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int a, b, r;
    printf("Ingrese el primer entero positivo: ");
    scanf("%d", &a);
    printf("Ingrese el segundo entero positivo: ");
    scanf("%d", &b);
    while (b != 0) {
        r = a % b;
        a = b;
        b = r;
    }
    printf("El MCD es %d\n", a);
    return 0;
}
```

Ejecución



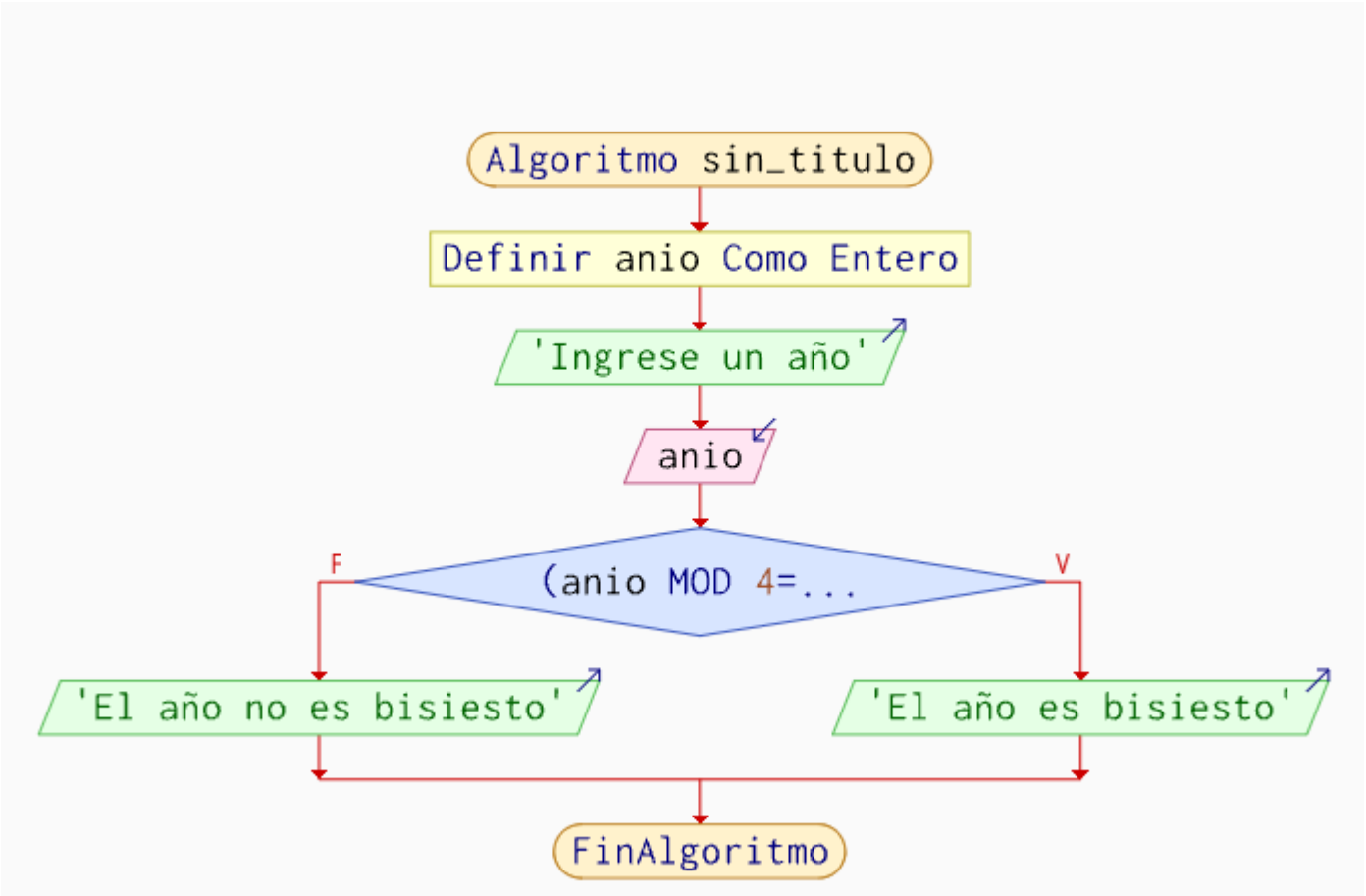
Problema 1.15. Año bisiesto
Entrada: año que se desea comprobar
Proceso: el ano bisiesto cumple si es visible entre 4 y no entre 100 tambien entre 400
Salida: mostrar si el año es bisiesto o no

Objeto	Nombre	Valor	Tipo
año	Año	Var	Entero
bisisesto	Bisiesto	var	logico

Seudocódigo

```
Algoritmo sin_titulo
    Definir anio Como Entero
    Escribir "Ingrese un año"
    Leer anio
    Si (anio MOD 4 = 0) Y ((anio MOD 100 ≠ 0) O (anio MOD 400 = 0)) Entonces
        Escribir "El año es bisiesto"
    Sino
        Escribir "El año no es bisiesto"
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c

#include <stdio.h>

int main() {
    int anio;

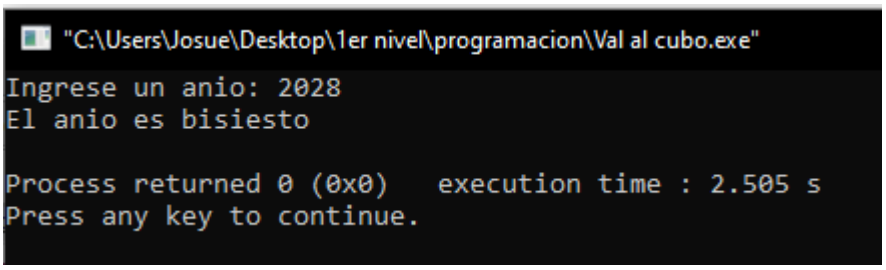
    printf("Ingrese un anio: ");
    scanf("%d", &anio);

    if ((anio % 4 == 0) && ((anio % 100 != 0) || (anio % 400 == 0))) {
        printf("El anio es bisiesto\n");
    } else {
        printf("El anio no es bisiesto\n");
    }

    return 0;
}

```

Ejecución



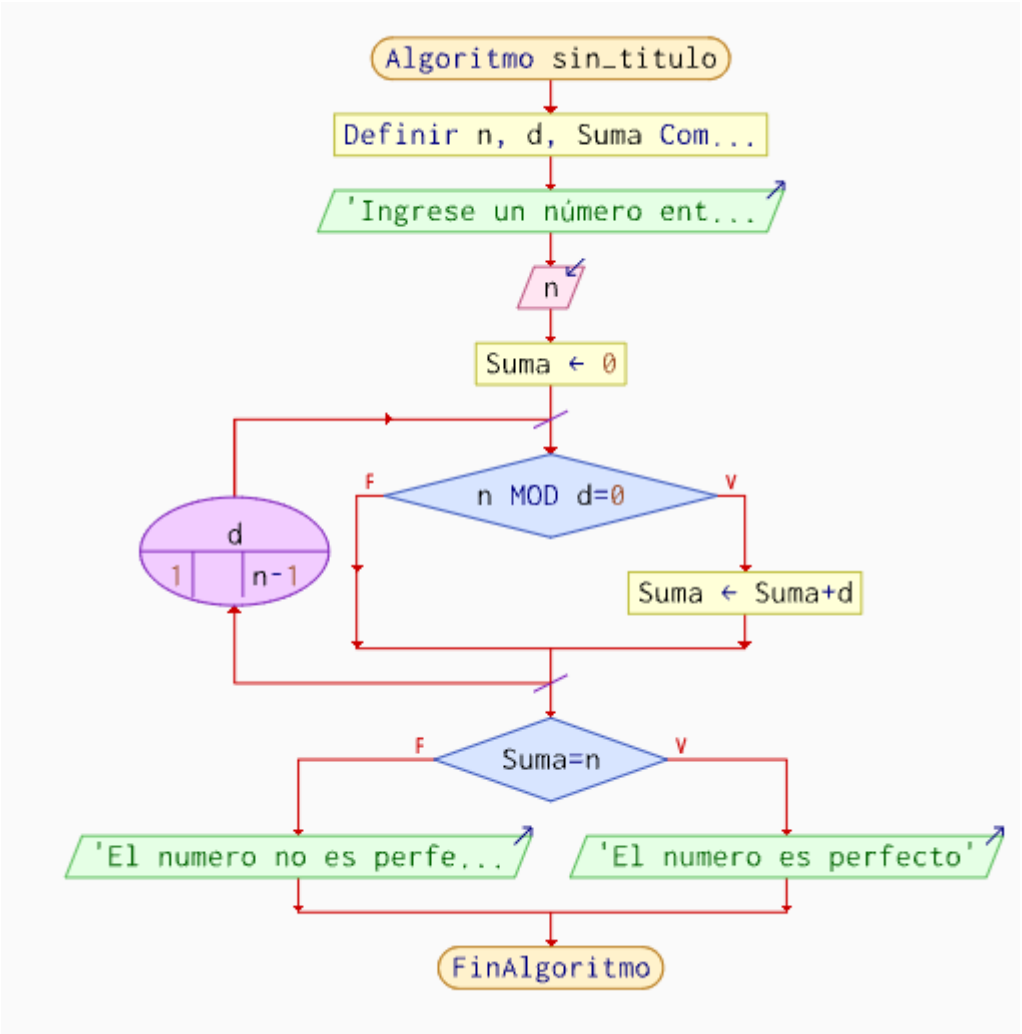
Problema 1.15. Año bisiesto
Entrada: numero entero positivo n
Proceso: leer inicializar la variable suma en 0 y recorrer las posibilidades desde 1 hasta n-1
Salida: indicar si el numero es perfecto o no

Objeto	Nombre	Valor	Tipo
N	N	Var	Entero
d	D	var	entero
suma	suma	Var	Entero

Seudocódigo

```
1  Algoritmo sin_titulo
2      Definir n, d, suma Como Entero
3      Escribir "Ingrese un número entero positivo"
4      Leer n
5      Suma ← 0
6      Para d ← 1 Hasta n - 1 Hacer
7          Si n MOD d = 0 Entonces
8              Suma ← Suma + d
9          FinSi
10     FinPara
11     Si Suma = n Entonces
12         Escribir "El numero es perfecto"
13     Sino
14         Escribir "El numero no es perfecto"
15     FinSi
16 FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int n, d, suma;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

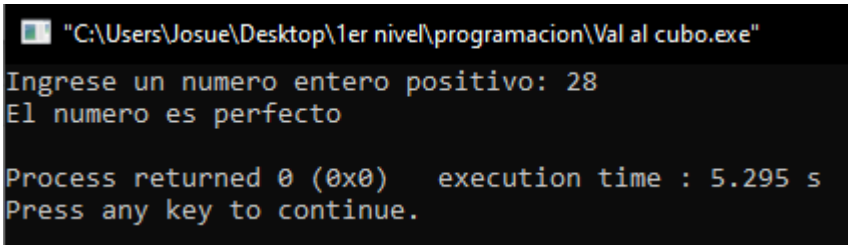
    suma = 0;

    for (d = 1; d <= n - 1; d++) {
        if (n % d == 0) {
            suma = suma + d;
        }
    }

    if (suma == n) {
        printf("El numero es perfecto\n");
    } else {
        printf("El numero no es perfecto\n");
    }

    return 0;
}
```

Ejecución



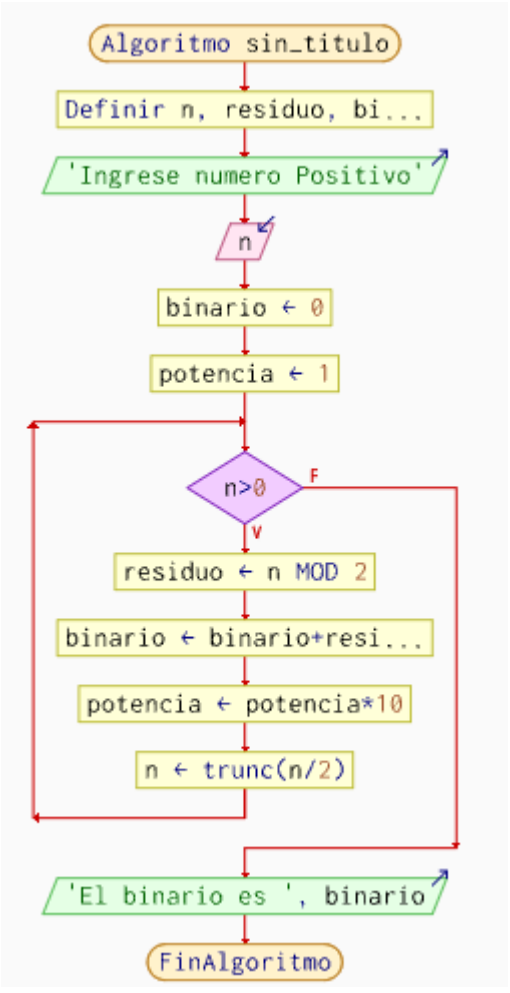
Problema 1.17. Conversión de decimal a binario
Entrada: n numero positivo decimal
Proceso: leer el numero n dividir entre dos y guardar los residuos
Salida: Muestra el numero en binario

Objeto	Nombre	Valor	Tipo
Numero decimal	n	Var	Real
Guarda el residuo entero	residuo	var	entero
Construye el binario	R	Var	Entero
p. del digito	p	var	entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir n, residuo, binario, potencia Como Entero
    Escribir "Ingrese numero Positivo"
    Leer n
    binario ← 0
    potencia ← 1
    Mientras n > 0 Hacer
        residuo ← n MOD 2
        binario ← binario + residuo * potencia
        potencia ← potencia * 10
        n ← trunc(n / 2)
    FinMientras
    Escribir "El binario es ", binario
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c

#include <stdio.h>

int main() {
    int n, residuo, binario, potencia;

    printf("Ingrese un numero positivo: ");
    scanf("%d", &n);

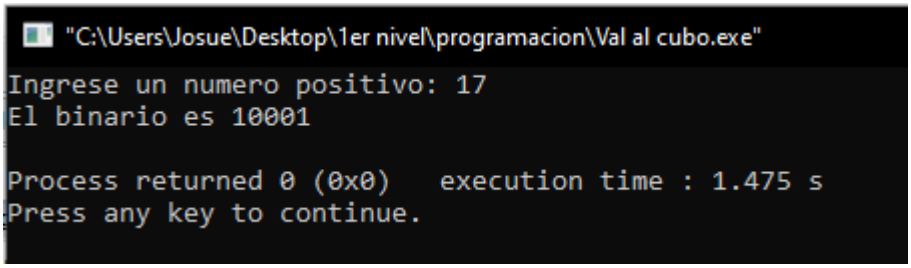
    binario = 0;
    potencia = 1;

    while (n > 0) {
        residuo = n % 2;
        binario = binario + residuo * potencia;
        potencia = potencia * 10;
        n = n / 2;
    }

    printf("El binario es %d\n", binario);

    return 0;
}
```

Ejecución



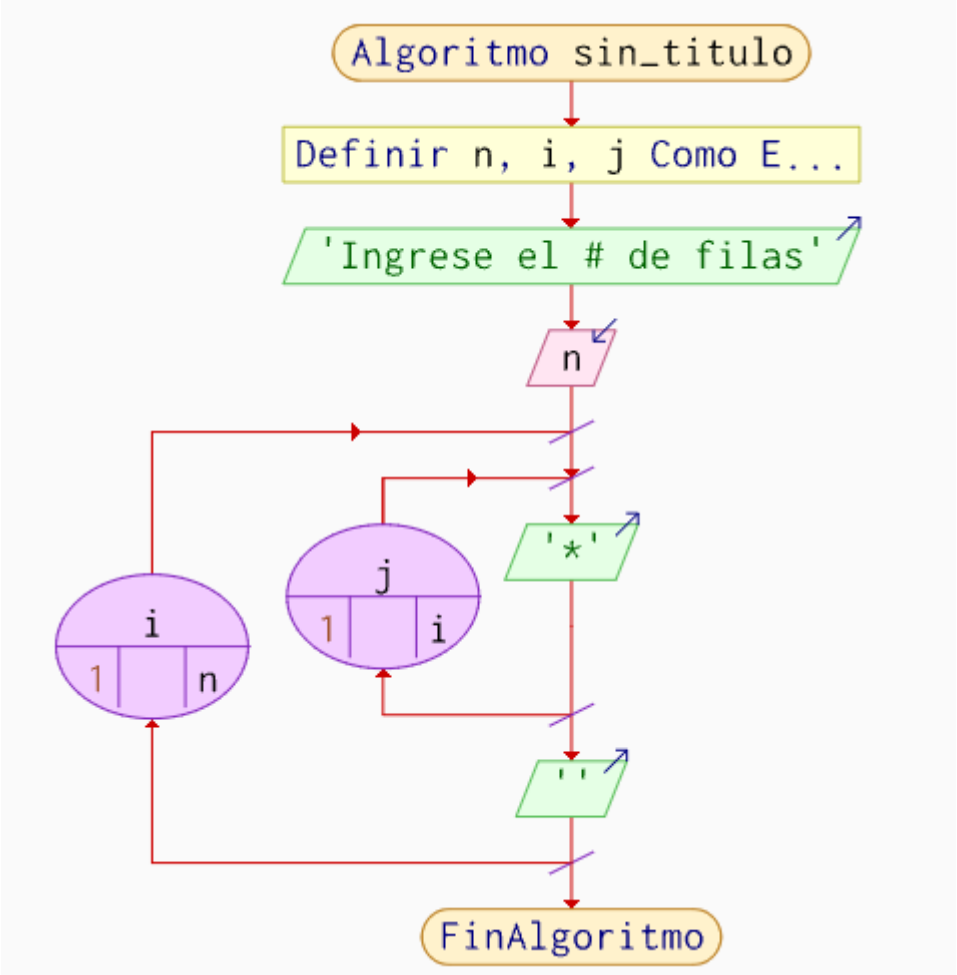
Problema 1.18. Triángulo de asteriscos
Entrada: leer n como numero de filas
Proceso: usar un ciclo para controlar filas de 1 hasta terminar cada fila un triangulo de *
Salida: imprime en pantalla el triangulo

Objeto	Nombre	Valor	Tipo
Numero de filas	N	Var	Real
Columnas	i	var	entero
Controlador *	j	Var	Entero

Seudocódigo

```
1  Algoritmo sin_titulo
2      Definir n, i, j Como Entero
3      Escribir "Ingrese el # de filas"
4      Leer n
5      Para i ← 1 Hasta n Hacer
6          Para j ← 1 Hasta i Hacer
7              Escribir Sin Saltar "*"
8          FinPara
9          Escribir ""
10     FinPara
11 FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Ingrese el numero de filas: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

Ejecución

```
"C:\Users\Josue\Desktop\1er nivel\programacion\Val al cubo.exe"
Ingrese el numero de filas: 5
*
**
***
****
*****

Process returned 0 (0x0)   execution time : 3.690 s
Press any key to continue.
```

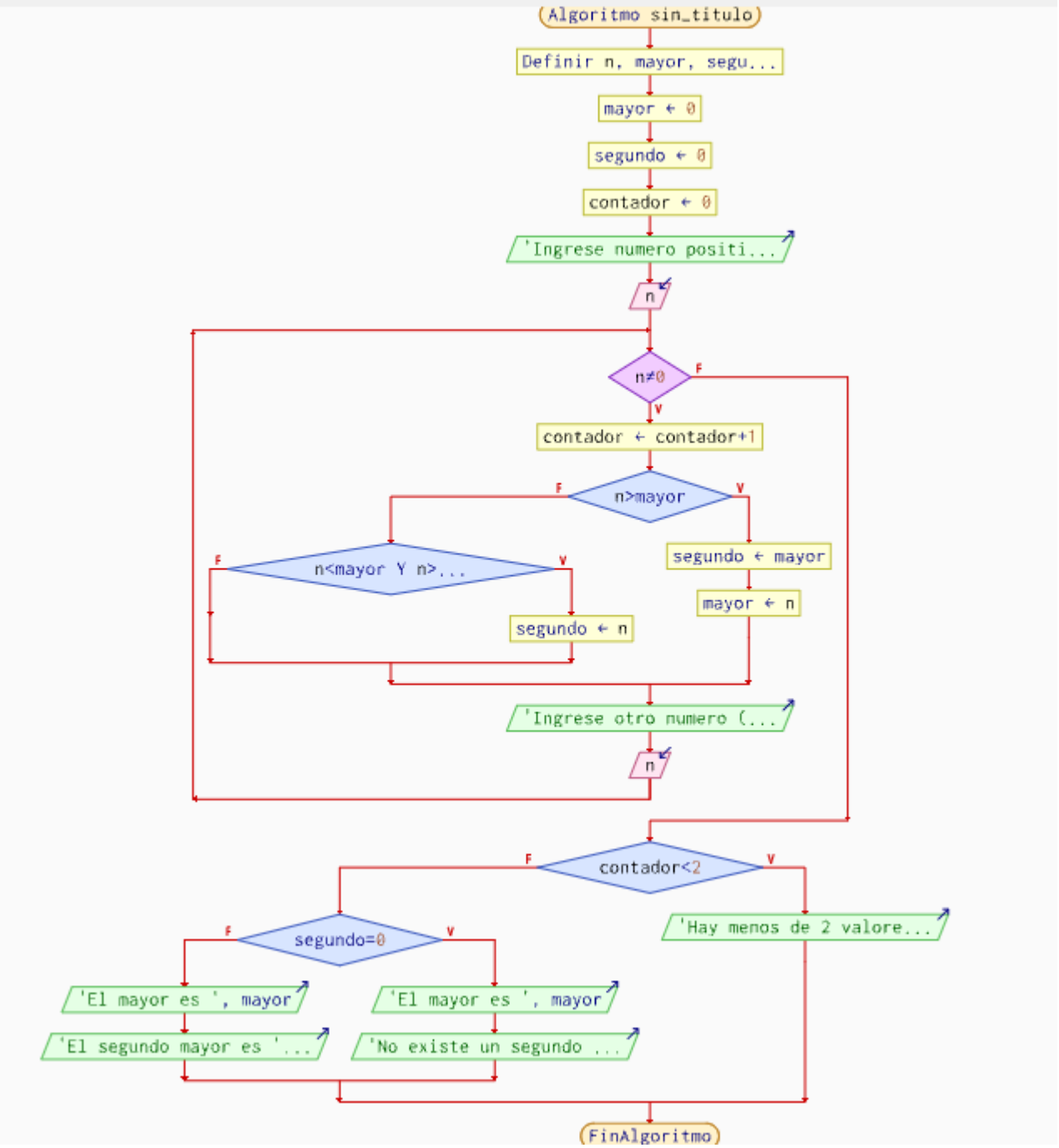
Problema 1.19. Segundo mayor de una secuencia
Entrada: secuencia de números positivos el 0 indica que termina la lectura
Proceso: leer los numero hasta que se ingrese el 0 y contar cuantos números se ingresaron
Salida: mostrar el mayor y el segundo mayor

Objeto	Nombre	Valor	Tipo
N	N	Var	Real
Mayor	Mayor	var	entero
Segundo	Segundo	Var	Entero
Contador	i	var	Entero

Seudocódigo

```
    segundo ← 0
    contador ← 0
    Escribir "Ingrese numero positivo (0 para terminar)"
    Leer n
    Mientras n ≠ 0 Hacer
        contador ← contador + 1
        Si n > mayor Entonces
            segundo ← mayor
            mayor ← n
        Sino
            Si n < mayor Y n > segundo Entonces
                segundo ← n
            FinSi
        FinSi
        Escribir "Ingrese otro numero (0 para terminar)"
        Leer n
    FinMientras
    Si contador < 2 Entonces
        Escribir "Hay menos de 2 valores ingresados"
    Sino
        Si segundo = 0 Entonces
            Escribir "El mayor es ", mayor
            Escribir "No existe un segundo mayor distinto"
        Sino
            Escribir "El mayor es ", mayor
            Escribir "El segundo mayor es ", segundo
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int n, mayor, segundo, contador;
    mayor = 0;
    segundo = 0;
    contador = 0;
    printf("Ingrese numero positivo (0 para terminar): ");
    scanf("%d", &n);
    while (n != 0) {
        contador = contador + 1;
        if (n > mayor) {
            segundo = mayor;
            mayor = n;
        } else {
            if (n < mayor && n > segundo) {
                segundo = n;
            }
        }
        printf("Ingrese otro numero (0 para terminar): ");
        scanf("%d", &n);
    }
    if (contador < 2) {
        printf("Hay menos de 2 valores ingresados\n");
    } else {
        if (segundo == 0) {
            printf("El mayor es %d\n", mayor);
            printf("No existe un segundo mayor distinto\n");
        } else {
            printf("El mayor es %d\n", mayor);
            printf("El segundo mayor es %d\n", segundo);
        }
    }
    return 0;
}
```

Ejecución

```
Ingrese numero positivo (0 para terminar): 2
Ingrese otro numero (0 para terminar): 4
Ingrese otro numero (0 para terminar): 6
Ingrese otro numero (0 para terminar): 9
Ingrese otro numero (0 para terminar): 45
Ingrese otro numero (0 para terminar): 1
Ingrese otro numero (0 para terminar): 2
Ingrese otro numero (0 para terminar): 5
Ingrese otro numero (0 para terminar): 6
Ingrese otro numero (0 para terminar): 47
Ingrese otro numero (0 para terminar): 456
Ingrese otro numero (0 para terminar): 486
Ingrese otro numero (0 para terminar): 1
Ingrese otro numero (0 para terminar): 2
Ingrese otro numero (0 para terminar):
4
Ingrese otro numero (0 para terminar): 40
Ingrese otro numero (0 para terminar): 0
El mayor es 486
El segundo mayor es 456

Process returned 0 (0x0)   execution time : 20.025 s
Press any key to continue.
```

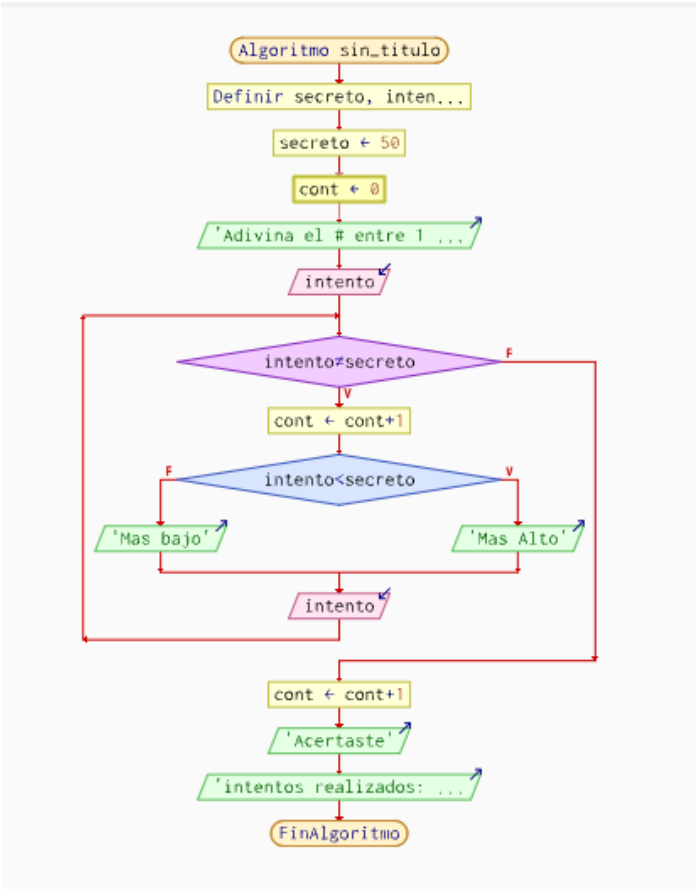
Problema 1.20. Adivina el número
Entrada: entrada de un numero que introduce el usuario
Proceso: estable un numero secreto fijo y pide números al usuario hasta que acierta
Salida: mostrar el mensaje de numero acertado y el numero de intentos

Objeto	Nombre	Valor	Tipo
intento	Intento	Var	Real
secreto	secreto	Var	entero
Contador	i	Var	Entero

Seudocódigo

```
Algoritmo sin_titulo
    Definir secreto, intento, cont Como Entero
    secreto ← 50
    cont ← 0
    Escribir "Adivina el # entre 1 y 100"
    Leer intento
    Mientras intento ≠ secreto Hacer
        cont ← cont + 1
        Si intento < secreto Entonces
            Escribir "Mas Alto"
        Sino
            Escribir "Mas bajo"
        FinSi
        Leer intento
    FinMientras
    cont ← cont + 1
    Escribir "Acertaste"
    Escribir "intentos realizados: ", cont
FinAlgoritmo
```

Diagrama de flujo



Código fuente

```c
#include <stdio.h>
int main() {
    int secreto, intento, cont;
    secreto = 50;
    cont = 0;
    printf("Adivina el numero entre 1 y 100: ");
    scanf("%d", &intento);
    while (intento != secreto) {
        cont = cont + 1;
        if (intento < secreto) {
            printf("Mas alto\n");
        } else {
            printf("Mas bajo\n");
        }
        scanf("%d", &intento);
    }
    cont = cont + 1;
    printf("Acertaste\n");
    printf("Intentos realizados: %d\n", cont);
    return 0;
}
```

Ejecución

